



دانشکده علوم ریاضی
دانشگاه فردوسی مشهد

پروژه دوم درس طراحی الگوریتم

افزایش پایداری شبکه با راهبرد حریصانه

استاد درس: دکتر رضا قنبری

اعضای گروه:

سارا مسگرمقدم ، اسما موحدی تبار

فهرست مطالب

۱	مقدمه و تعریف مسئله	۲
۲	رابطه‌ی یال‌ها و برش‌ها	۲
۳	تعریف دقیق معیار حریصانه	۲
۴	شرح الگوریتم گام‌به‌گام	۳
۵	تحلیل کد پیاده‌سازی شده	۴
۱.۵	ساخت گراف کمکی	۴
۲.۵	یافتن دو طرف برش با DFS	۴
۳.۵	انتخاب ارزان‌ترین کابل پوشش‌دهنده	۵
۴.۵	مکانیزم استفاده‌ی رایگان	۵
۶	عدم بهینگی راهبرد حریصانه	۶

۱ مقدمه و تعریف مسئله

درخت پوشای کمینه (MST) که در فاز اول ساخته شد، هسته‌ی اصلی شبکه‌ی مخابراتی امپراتوری را تشکیل می‌دهد. اما ساختار درختی یک ضعف حیاتی دارد: از آنجا که بین هر دو راس دقیقاً یک مسیر وجود دارد، قطع شدن هر یک از یال‌های آن، شبکه را به دو مؤلفه‌ی مجزا تقسیم می‌کند.

با فرض اینکه دقیقاً یکی از یال‌های MST (که از پیش مشخص نیست کدام یک) توسط طوفان قطع خواهد شد، هدف این فاز، انتخاب زیرمجموعه‌ای از کابل‌های پشتیبان پیشنهادی است به گونه‌ای که برای هر یال از MST، حداقل یک کابل پشتیبان وجود داشته باشد که در صورت قطع آن یال، دو بخش جداشده را مجدداً به هم متصل کند. این کار باید با کمترین هزینه‌ی ممکن و با استفاده از یک راهبرد حریصانه انجام شود.

۲ رابطه‌ی یال‌ها و برش‌ها

در یک درخت، حذف هر یال آن را دقیقاً به دو مؤلفه‌ی همبند تقسیم می‌کند. این تقسیم را در نظریه گراف یک برش (Cut) می‌نامند. یک کابل پشتیبان (a, b) تنها زمانی می‌تواند نقش یال حذف‌شده را ایفا کند که یک سر آن (a) در یکی از این دو مؤلفه و سر دیگر آن (b) در مؤلفه‌ی دیگر قرار داشته باشد. به عبارت دیگر:

این مشاهده مبنای اصلی الگوریتم پیاده‌سازی شده است: برای هر یال MST، ابتدا باید دو طرف برش را با پیمایش گراف مشخص کرد، سپس در میان کابل‌های پشتیبان، آن‌هایی را که این برش را می‌پوشانند شناسایی نمود.

۳ تعریف دقیق معیار حریصانه

راهبرد حریصانه‌ی پیاده‌سازی شده به شرح زیر است:

برای هر یال e از MST به‌طور مستقل و بدون در نظر گرفتن تصمیمات آینده، ارزان‌ترین کابل پشتیبان هنوز انتخاب نشده‌ای که برش ناشی از حذف e را می‌پوشاند انتخاب می‌شود. در صورتی که یک کابل پشتیبان که پیش‌تر برای یال دیگری انتخاب شده است، به‌طور اتفاقی برش فعلی را نیز بپوشاند، استفاده مجدد از آن رایگان محسوب شده و نیازی به انتخاب کابل جدید نیست.

این معیار یک الگوریتم حریصانه‌ی یال‌محور است: تصمیم‌گیری به ازای هر یال از MST به‌صورت جداگانه و بر اساس بهترین انتخاب محلی (ارزان‌ترین گزینه‌ی موجود در آن لحظه) صورت می‌گیرد، بدون آنکه امکان بهره‌برداری هم‌زمان یک کابل از چند برش به‌طور پیشینی در تصمیم‌گیری لحاظ شود (به‌جز از طریق مکانیزم «استفاده‌ی رایگان» که پس از تصمیم اولیه اعمال می‌شود).

چرا این معیار حریصانه است؟ چون تصمیم فقط بر اساس وضعیت فعلی گرفته می‌شود. الگوریتم بررسی نمی‌کند که: آیا این کابل در آینده برای یال‌های دیگر هم مفید است؟ آیا انتخاب کابل دیگری باعث کاهش هزینه کل می‌شود؟ آیا یک کابل گران‌تر می‌تواند چندین یال را همزمان پوشش دهد؟ بلکه فقط بهترین انتخاب در همان لحظه را انجام می‌دهد.

چرا این معیار را انتخاب کردیم؟ این معیار انتخاب شد زیرا: پیدا کردن ارزان‌ترین کابل مناسب در هر مرحله ساده است. تصمیم‌گیری فقط به اطلاعات همان مرحله وابسته است. نیاز به بررسی همه حالت‌های ممکن ندارد. زمان اجرای الگوریتم نسبت به روش‌های جستجوی کامل کمتر است. البته این معیار تنها بهینه محلی را انتخاب می‌کند و تضمینی برای رسیدن به کمترین هزینه کل ندارد.

۴ شرح الگوریتم گام به گام

الگوریتم در پنج گام کلی اجرا می‌شود:

۱. ساخت ساختار درخت MST: یال‌های ورودی به یک لیست مجاورت تبدیل می‌شوند تا امکان پیمایش گراف فراهم شود.

۲. پیمایش برای هر یال: به ازای هر یال (u, v) از MST با DFS از راس u آغاز می‌شود، با این تفاوت که عبور از یال (u, v) ممنوع است. مجموعه‌ی راس‌های قابل دسترسی از u (که مجموعه‌ی sideU نامیده می‌شود) به این ترتیب به دست می‌آید؛ سایر راس‌ها به‌طور ضمنی طرف v محسوب می‌شوند.

۳. جست‌وجوی ارزان‌ترین پوشش‌دهنده: در میان کابل‌های پشتیبان هنوز استفاده نشده، آن‌هایی که یک سرشان در sideU و سر دیگرشان خارج از آن باشد (یعنی $aInU \neq bInU$) شناسایی و ارزان‌ترین آن‌ها انتخاب می‌شود.

۴. بررسی استفاده‌ی رایگان یا شکست: اگر هیچ کابل استفاده نشده‌ای یافت نشود، ابتدا بررسی می‌شود که آیا یکی از کابل‌های قبلاً انتخاب شده به‌طور اتفاقی این برش را نیز می‌پوشاند؛ اگر چنین کابلی وجود داشته باشد، بدون افزودن هزینه‌ی جدید ادامه داده می‌شود. در غیر این صورت، مقاوم‌سازی برای این یال (و در نتیجه برای کل شبکه) غیرممکن اعلام می‌شود.

۵. انباشت هزینه و خروجی: کابل انتخاب شده علامت‌گذاری، هزینه‌اش به مجموع اضافه و به لیست خروجی افزوده می‌شود. این فرآیند برای تمام یال‌های MST تکرار می‌شود.

۵ تحلیل کد پیاده‌سازی شده

۱.۵ ساخت گراف کمکی

```

۱ for(int[] edge : mstEdges){
۲     int u = edge[0], v = edge[1];
۳     mstAdj.get(u).add(v);
۴     mstAdj.get(v).add(u);
۵ }

```

از آنجا که MST یک گراف بدون جهت است، هر یال به هر دو راس دو سرش در لیست مجاورت اضافه می‌شود. این ساختار داده امکان پیمایش کارآمد گراف را برای گام بعدی فراهم می‌کند.

۲.۵ یافتن دو طرف برش با DFS

```

۱ Set<Integer> sideU = new HashSet<>();
۲ Deque<Integer> stack = new ArrayDeque<>();
۳ stack.push(u);
۴ sideU.add(u);
۵ while(!stack.isEmpty()){
۶     int cur = stack.pop();
۷     for(int next : mstAdj.get(cur)){
۸         if((cur == u && next == v) || (cur == v && next == u)) continue;
۹         if(!sideU.contains(next)){
۱۰             sideU.add(next);
۱۱             stack.push(next);
۱۲         }
۱۳     }
۱۴ }

```

این بخش یک پیاده‌سازی تکرارشونده (غیر بازگشتی) از DFS با استفاده از پشته (Stack) است. استفاده از HashSet برای sideU امکان بررسی عضویت راس در مجموعه را در زمان میانگین $O(1)$ فراهم می‌کند، که در مقایسه با استفاده

از یک لیست، کارایی الگوریتم را به‌طور محسوسی بهبود می‌بخشد. شرط `continue` در ابتدای حلقه، دقیقاً همان یالی را که در حال بررسی حذف آن هستیم نادیده می‌گیرد و مانع از عبور DFS از آن می‌شود؛ این کار به‌طور مؤثر همان اثر «حذف موقت یال» را بدون تغییر واقعی ساختار داده شبیه‌سازی می‌کند.

۳.۵ انتخاب ارزان‌ترین کابل پوشش‌دهنده

```

۱ for(int i = 0; i < m; i++){
۲     if (usedBackup[i]) continue;
۳     boolean aInU = sideU.contains(a);
۴     boolean bInU = sideU.contains(b);
۵     if(aInU != bInU){
۶         if(c < bestCost){ bestCost = c; bestIdx = i; }
۷     }
۸ }
```

عبارت `aInU != bInU` معادل منطقی بررسی «آیا این دو راس در دو طرف مختلف برش قرار دارند؟» است؛ این عبارت تنها زمانی درست است که دقیقاً یکی از دو مقدار بولی برابر `true` باشد، که دقیقاً معادل قرارگیری در دو مؤلفه‌ی متفاوت است.

۴.۵ مکانیزم استفاده‌ی رایگان

```

۱ if(bestIdx == 1-){
۲     boolean anyCoverExists = false;
۳     for(int i = 0; i < m; i++){
۴         ...
۵         if(aInU != bInU){ anyCoverExists = true; break; }
۶     }
۷     if(!anyCoverExists){ possible = false; break; }
۸     continue;
۹ }
```

این بخش یکی از ظرافت‌های پیاده‌سازی است. وقتی هیچ کابل استفاده‌نشده‌ای برش فعلی را پوشش ندهد، بررسی

می‌شود که آیا در میان تمام کابل‌ها (اعم از استفاده‌شده یا نشده) کابلی برای این برش وجود دارد یا نه. اگر بله، یعنی یک کابل قبلاً برای یال دیگری انتخاب شده و به‌طور اتفاقی این یال را نیز پوشش می‌دهد؛ در این حالت، آن کابل رایگان محسوب شده و از دوبار محاسبه‌ی هزینه‌ی آن جلوگیری می‌شود. اگر حتی در میان تمام کابل‌ها نیز پوشش‌دهنده‌ای یافت نشود، مسئله برای این یال غیرقابل حل بوده و کل خروجی NO خواهد بود.

۶ عدم بهینگی راهبرد حریصانه

لازم به ذکر است که راهبرد حریصانه‌ی معرفی‌شده در این فاز، بهینگی را تضمین نمی‌کند. علت این امر آن است که تصمیم‌گیری برای هر یال به‌صورت مستقل و بر اساس بهترین گزینه‌ی محلی صورت می‌گیرد، در حالی که ممکن است یک کابل با هزینه‌ی نسبتاً بالاتر، به‌طور هم‌زمان چند یال را پوشش دهد و در مجموع اقتصادی‌تر از چند انتخاب مستقل ارزان‌تر باشد. یک مثال نقض دقیق برای این ادعا، همراه با اثبات ریاضی و مقایسه‌ی عددی با جواب بهینه، در گزارش فاز سوم (که از روش شاخه و کران استفاده می‌کند) ارائه شده است.